

## Organizing Assembly Module Procedures

1. Before the program entry point (i.e., before the first instruction of a program)
2. After the last instruction of a program entry point (i.e., after instruction that ends the program)
3. Within the other procedure (must guarantee that there are JMP instruction that passed over such procedure)
4. In an other module using:  
    PUBLIC name\_1, ..., name\_n  
    and  
    EXTRN name\_1: type\_1, ..., name\_n: type\_n

Rīgas Tehniskā universitāte

Computer Organization and Assembly Language

1

The main reason for assembly module organization is to warrant against malicious start of a procedure execution. The first three options assumes that the procedures resides on the same code segment, while last assumes that the code segment might be different.

Because each module might be implemented in own programming language/technology, there must be assumptions how to ensure control and data communication between such modules. Especially it becomes important when assembly modules must communicate with high-level languages (HLL) modules (e.g., C/C++) and vice versa.

Because of modularity principle, the module must not know anything about the internals of other modules. However, to ensure communication, there must be some interface via which it can happen. For this reason, assembler provides a means to define some interface procedure or variable as public, thus ensuring its visibility from other modules. Therefore, if some module requires access to such procedure/variable, there is a way to tell the assembler that it must be aware of such situation.

To provide such inter-module visibility, in the assembler code procedure or variable must be declared with directive PUBLIC. Then from other modules it can be referenced after using directive EXTRN.

For correct referencing of external variables, its type must be specified (e.g., BYTE, WORD, DWORD, etc.).

# Example of Using External Procedures

```
; Module 1:
.686
.model flat, stdcall
.code
:::
ProcName1 PROC
    call ProcName2
:::
ProcName1 ENDP
ProcName2 PROC PUBLIC
:::
ProcName2 ENDP
END
```

Declare procedure  
ProcName2 as being  
externally accessible.

```
; Module 2:
.686
.model flat, stdcall
.code
EXTRN ProcName2
start:
    call ProcName2
    END start
```

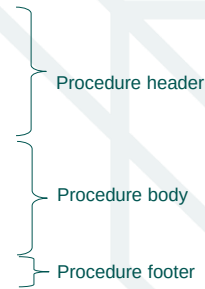
Reference externally  
accessible procedure.

Call procedure  
ProcName2

# MASM Procedure Definition

The extended syntax of procedure directive (MASM specific):

```
proc_name PROC [distance] [langtype] [visibility]
    [USES register_1 register_2 ...] [argument_1: type_1,
    argument_2: type_2, ...]
    [LOCAL label_1[[count]]: type_1, ...]
    :::
    :::
    :::
proc_name ENDP
```



When passing of procedure parameters over the stack, the assembler provides additional options that can be used during the definition of a procedure.

Optional *distance* usually can be either NEAR or FAR.

Optional *langtype* defines the calling convention used in HLL and a public name mangling.

Optional *visibility* can be either PRIVATE, PUBLIC (by default) or EXPORT.

If procedure requires arguments that have to be passed via the stack, then it is possible to declare them explicitly. This allows in the procedure body to use specified arguments by their name (instead of explicitly relating to EBP register).

If procedure modifies general purpose registers, then listing them after USES allows assembler automatically save their values into the stack at the procedure entry and restore them upon exit.

Finally, if the procedure have to use additional variables that are used only while procedure instructions are executing, then they can be specified after the LOCAL. For each such variable, optional <count> and the type can be specified. By using this approach, the assembler automatically will place instruction to clean up the stack from passed and local variables.

# Assembler Procedure vs. Generated Instructions

```
Bin2Txt PROC USES EAX EBX ECX EDX ESI EDI \
    num: DWORD, buf: PTR BYTE
;Procedure Bin2Txt - converts 32-bit value
to ASCII text
;Input:    num - 32-bit number to convert
;          buf - buffer for output text
;          LOCAL tmp[10]: BYTE

    :::

    ret
Ones ENDP
```

```
Bin2Txt PROC USES EAX EBX ECX EDX ESI EDI \
    wrd: DWORD, buf: PTR BYTE
00F9154A push ebp
00F9154B mov  ebp, esp
00F9154D add  esp, -12
00F91550 push eax
00F91551 push ebx
00F91552 push ecx
00F91553 push edx
00F91554 push esi
00F91555 push edi
    :::
00F91589 pop  edi
00F9158A pop  esi
00F9158B pop  edx
00F9158C pop  ecx
00F9158D pop  ebx
00F9158E pop  eax
00F9158F leave
00F91590 ret
Ones ENDP
```

-12 (and not -10)  
because of  
memory  
alignment to  
DWORD  
boundary!

the instruction LEAVE  
is equivalent to:  
MOV ESP, EBP  
POP EBP

Rīgas Tehniskā universitāte

Computer Organization and Assembly Language

4

```
Bin2Txt PROC USES EAX EBX ECX EDX ESI EDI \
    num: DWORD, buf: PTR BYTE
;Procedure Bin2Txt - converts 32-bit value to ASCII text
;Input:    num - 32-bit number to convert
;          buf - buffer for output text
;          LOCAL tmp[10]: BYTE ; An array of ten bytes
    :::
    ret
Ones ENDP
```

As you can see, the use of various PROC options allows to simplify assembly procedure definition – the assembler automatically adds additional instructions according to options (shown in red).

# High Level Language Parameter Passing Conventions

| Language Type       | Argument Direction | Stack Clean-up Responsibility |
|---------------------|--------------------|-------------------------------|
| C                   | ←                  | Caller                        |
| STDCALL             | ←                  | Callee                        |
| FASTCALL / REGISTER | →                  | Callee                        |

```
extern "C" void __cdecl foo1(int a1, double a2, ...) { :: }
extern "C" void __stdcall foo2(int a1, double a2, char * a3) { :: }
extern "C" void __fastcall foo3(int a1, double a2, char * a3) { :: }

int main() {
    foo1(1, 2.0, "abc");
    foo2(1, 2.0, "abc");
    return 0;
}
```

Rīgas Tehniskā universitāte

Computer Organization and Assembly Language

5

Historically each specific HLL have developed own scheme of parameter passing and stack management. Therefore, depending on module communication agreement with other modules, the language type option becomes an important parameter. It influences the way the procedure parameters will be pushed into the stack and how the stack will be cleaned-up after the call. In addition, the language type also defines the instructions that will be generated by the assembler in the procedure *prologue* and *epilogue*.

The default language type at the module level is specified with the assembler directive `.MODEL` but can be overridden for each procedure individually.

The table shows only the most often used calling conventions – there might be other language types and conventions available.

Procedure argument passing direction specifies, which parameter is pushed first on the stack. It is the last one for C and STDCALL conventions, and the first for FASTCALL conventions.

STDCALL is the default convention used for interfacing with Windows APIs.

Note that only C calling conventions allows to define/call variable parameter count procedures. Thus, only for them it possible to define procedure argument with the type `VARARG`.

In addition to calling and stack clean-up conventions, the language type also specifies the name mangling style by which PUBLIC procedures and variables are made visible to other modules and HLLs.

Some conventions over time have become obsolete (e.g., SYSCALL and PASCAL).

For more, see: <https://learn.microsoft.com/en-us/cpp/cpp/argument-passing-and-naming-conventions>.

# Writing Assembly Language Module to Interface with HLL

```
.686
.model flat, C
.code
Ones PROC USES EAX EBX ECX wrd : DWORD, count : BYTE
;Procedure Ones - counts the number of ones in the rightmost bits
;Input: wrd - number in which bits to be counted
; count - number of rightmost bits to count
;Output: EAX - the number of counted bits
    movzx ecx, [count]
    mov  eax, [wrd]
    xor  ebx, ebx
ones_loop:
    test eax, 1
    jz   skip_inc
    inc  ebx
skip_inc:
    shr  eax, 1
    loop ones_loop
    mov  eax, ebx
    ret
Ones ENDP
:::
```

Rīgas Tehniskā universitāte

Computer Organization and Assembly Language

6

This example shows bit count calculation procedure that was rewritten from Inline Assembler to assembler procedure and placed in a separate .ASM module file.

## Writing Assembly Language Module to Interface with HLL, cont. 1

```
; continues from the previous slide
Bin2Txt PROC USES EAX EBX ECX EDX ESI EDI \
    wrd: DWORD, buf: PTR BYTE
;Procedure Bin2Txt - converts 32-bit value
to ASCIIZ string
;Input:  wrd - number to convert
;        buf - text buffer

    LOCAL tmp[10]: BYTE

    mov eax, [wrd]
    lea esi, [tmp]
    mov ecx, 10

div_loop:
    cdq
    div ecx
    add dl, 30h ; an make ASCII digit
    mov BYTE PTR [esi], dl
    cmp eax, 0
    je converted
    inc esi
    jmp div_loop
converted:

; continues from the left
    mov edi, [buf]
    lea ebx, [tmp]
    ; ESI points at the end of tmp[]
    mov ecx, esi
    inc ecx
    sub ecx, ebx
    jecxz exit

rev_loop:
    ; copy to buf in the reverse order
    mov al, [esi]
    mov [edi], al
    dec esi
    inc edi
    loop rev_loop

exit:
    mov BYTE PTR [edi], 0
    ret

Bin2Txt ENDP

END ; end of the module
```

This example continues the previous slide and shows binary unsigned integer conversion to the ASCIIZ string.

# Writing Assembly Language Module to Interface with HLL, cont. 2

Calling assembly language procedures from C/C++:

```
#include <stdio>
```

*extern "C" tells C++ compiler that this function (callee) expects the C calling convention.*

```
extern "C" int Ones(unsigned wrd, char count);  
extern "C" void Bin2Txt(unsigned wrd, char buf[]);
```

```
int main(void)  
{  
    char buffer[] = "*****";  
    Bin2Txt(Ones(1234, 10), buffer);  
    printf("Hello %s\n", buffer);  
    getchar();  
    return 0;  
}
```



## Writing Assembly Language Module to Interface with HLL, cont. 3

The disassembly of C/C++ compiler generated code:

```
Bin2Txt(Ones(1234, 10), buffer);
```

```
lea     eax, [ebp-14h] ; the address of array b[]
```

```
push    eax
```

```
push    0Ah
```

```
push    4D2h
```

```
call    _Ones
```

```
add     esp, 8
```

```
push    eax
```

```
call    _Bin2Txt
```

```
add     esp, 8
```

Parameter passing order – from right to left.

The C calling convention mangles the names of public procedures.

Under the C calling convention, the caller must clean-up the stack.

```
printf("Hello %s\n", buffer);
```

```
lea     eax, [ebp-14h]
```

```
push    eax
```

```
push    offset string "Hello %s\n" (1355800h)
```

```
call    dword ptr [__imp_printf]
```

```
add     esp, 8
```

## Writing Assembly Language Module to Interface with HLL, cont. 4

- How the C calling convention passes parameters via the stack in IA-32 Windows OS:
  - For C/C++ built-in ordinal types (char, short, int, unsigned, long) – as DWORD value.
  - For 64-bit types (long long) – as two DWORD values.
  - For C/C++ built-in floating-point types (float, double) – via FPU register.
  - For pointers to built-in types and arrays – as PTR <type> (i.e., as DWORD value).
- How C calling convention returns values in IA-32:
  - For C/C++ built-in ordinal types:
    - bool, char – in AL register
    - short – in AX register
    - int, unsigned, long – in EAX register
    - long long – in EDX:EAX register pair
  - For C/C++ built-in floating-point types (float, double) – via FPU register.
  - For type pointers – via EAX register.

This as a reference that can be used when you have to call HLL functions from the IA-32 assembler or create assembler procedures to be called from HLLs in Windows.

# Creating Windows Dynamic-Link Library in Assembly Language

```
.586
.model flat, C
.code

Ones PROC EXPORT \
    USES EBX ECX \
    wrd: DWORD, count: BYTE

    :::

Ones ENDP

Bin2Txt PROC EXPORT \
    USES EAX EBX ECX EDX ESI EDI \
    wrd: DWORD, buf: PTR BYTE

    :::

Bin2Txt ENDP

END
```

Rīgas Tehniskā universitāte

Computer Organization and Assembly Language

11

A Windows *dynamic-link library* (DLL) is a file module that contains functions and data that can be used by another module (application or DLL).

A DLL can define two kinds of functions: exported and internal. The exported functions are intended to be called by other modules, as well as from within the DLL where they are defined. Internal functions are intended to be called only from within the DLL where they are defined. Although a DLL can export data, its data is generally used only by its functions. However, there is nothing that prevents another module from reading or writing that data.

DLLs provide a way to modularize applications so that their functionality can be updated and reused more easily. DLLs also help to reduce memory overhead when several applications use the same functionality at the same time, because although each loaded user application receives its own copy of the DLL data, the applications share the DLL code.

Every process that loads the DLL maps it into its virtual address space. After the process loads the DLL into its virtual address, it can call the exported DLL functions.

A DLL can optionally specify an entry-point function. If present, the system calls the entry-point function whenever a process or thread loads or unloads the DLL. It can be used, for example, to perform simple initialization and cleanup tasks.

For more, see: <https://learn.microsoft.com/lv-lv/windows/win32/dlls/dynamic-link-libraries>.

# Using Windows Dynamic-Link Library from C/C++ Executable App

```
#include <stdio>

extern "C" int Ones(unsigned wrd, char count);
extern "C" void Bin2Txt(unsigned wrd, char * buf);

int main() {
    char buffer[] = "*****";
    Bin2Txt(Ones(55, 5), buffer); //0011 0111
    printf("%s", buffer);
    getchar();
    return 0;
}
```

```
'[.].exe' (Win32): Loaded 'C:\[.].Debug\[.].exe'. Symbols loaded.
'[.].exe' (Win32): Loaded 'C:\Windows\SysWOW64\ntdll.dll'.
'[.].exe' (Win32): Loaded 'C:\Windows\SysWOW64\kernel32.dll'.
'[.].exe' (Win32): Loaded 'C:\Windows\SysWOW64\KernelBase.dll'.
'[.].exe' (Win32): Loaded 'C:\[.].Debug\[.].DLL.dll'. Symbols loaded.
'[.].exe' (Win32): Loaded 'C:\Windows\SysWOW64\vcruntime140d.dll'.
'[.].exe' (Win32): Loaded 'C:\Windows\SysWOW64\ucrtbased.dll'.
```

Rīgas Tehniskā universitāte

Computer Organization and Assembly Language

12

An executable file links to (or loads) a DLL in one of two ways:

- Implicit linking is sometimes referred to as static load or load-time dynamic linking.
- Explicit linking is sometimes referred to as dynamic load or run-time dynamic linking.

With implicit linking, the executable using the DLL links to an import library (.lib) file provided by the maker of the DLL. The operating system loads the DLL when the executable using it is loaded. The client executable calls the DLL's exported functions just as if the functions were contained within the executable.

With explicit linking, the executable using the DLL must make function calls to explicitly load and unload the DLL and to access the DLL's exported functions. The client executable must call the exported functions through a function pointer.

An executable can use the same DLL with either linking method. Furthermore, these mechanisms are not mutually exclusive, as one executable can implicitly link to the DLL and another can attach to it explicitly.

For configuring the DLL project as a dependency to the Console App project, in Visual Studio 2022 right click on the References node and from the context menu select Add Reference... and mark the checkbox for the DLL project.

In older Visual Studio versions, for linking executable application project with DLLs in Visual Studio consult, for example:

<http://stackoverflow.com/questions/7683869/what-does-the-link-library-dependency-linker-option-actually-do-in-visual-stud>:

Common Properties -> Framework and References : (List of dependencies)

Each lib/dll has a separate option: Project Reference Properties -> Link Library Dependencies : True|False